

# **Hochschule Bonn Rhein Sieg**

Fachbereich Informatik

## **Seminararbeit**

im Studiengang Informatik mit Vertiefung Informationssicherheit B.Sc

Thema:

## **Reduktion der Zertifikatsgröße in TLS-Verbindung**

vorgelegt von:

Bahadir Hacisalihoglu  
Matrikelnummer: 9053141

Abgabedatum: 13.01.2026

# Inhaltsverzeichnis

|                                                                 |           |
|-----------------------------------------------------------------|-----------|
| <b>1.Einleitung.....</b>                                        | <b>3</b>  |
| <b>2.Erläuterung der Ansätze.....</b>                           | <b>4</b>  |
| 2.1 Zertifikatskompression (RFC 8879).....                      | 4         |
| 2.2 Client Certificate URLs (RFC 6066).....                     | 7         |
| 2.3 Cached Information Extension (RFC 7924).....                | 8         |
| 2.4 CBOR-encoded Certificates (I-D.ietf-cose-cbor-encoded-cert) | 9         |
| 2.5 Raw Public Keys (RFC 7250).....                             | 11        |
| 2.6 cTLS Certificate Handles (I-D.ietf-tls-ctls).....           | 13        |
| <b>3. Fazit.....</b>                                            | <b>14</b> |
| <b>4. Literaturverzeichnis.....</b>                             | <b>16</b> |

## 1 Einleitung

TLS-Zertifikate machen einen erheblichen Teil des Datenverkehrs in durch TLS/SSL gesicherten Verbindungen aus. Typischerweise bewegen sich die Größen von im Internet eingesetzten X.509 Zertifikaten(RFC 5280) im Bereich zwischen 500 und 1 500 Byte (vgl. Kampanakis et al., 2018). Ein X.509-Zertifikat von der Webseite <https://www.example.com> hat eine Größe von 1 526 Byte.

Diese Zertifikatsnachrichten machen 61,5 % des Bandbreiten-Overheads von TLS 1.2 und 57,9 % bei TLS 1.3 aus (vgl. Bodenhausen et al., 2025).

Mit zunehmender Schlüssellänge vergrößert sich auch die Größe der entsprechenden Zertifikate. Zum Beispiel die öffentlichen Schlüssel und Signaturen von Post-Quanten-Verfahren liegen in der Regel zwischen 10 und 20 KB, während RSA-Verfahren deutlich kleinere Schlüssel- und Signaturlängen haben.

Studien zeigen, dass die Post-Quantum-Kryptografie derzeit noch nicht weit verbreitet ist, und in Zukunft ein großes Wachstumspotenzial haben.

In einer Messstudie von Jakub Sowa et al. wurde am National Center for Supercomputing Applications keine organisatorische Implementierung von Post-Quantum-Kryptografie im TLS-Protokoll nachgewiesen. (vgl. Sowa et al., 2024).

Ein weiterer Aspekt, der die Bedeutung von Ansätzen zur Verringerung der übertragenen Zertifikatsgröße unterstreicht, betrifft IoT-Geräte und Drahtlose Netzwerke. Aufgrund ihrer geringen Rechenleistung ist die Optimierung des TLS-Handshakes für sie besonders wichtig.

Große Zertifikate erfordern mehr Speicher zur Verarbeitung was für speicherbeschränkte IoT-Geräte herausfordernd ist.

Drahtlose Netzwerke wie LoRaWAN haben maximale Nachrichtengrößen von nur 256 Bytes, was die Übertragung großer Zertifikate praktisch unmöglich macht (vgl. Forsby et al., 2017).

Vor diesem Hintergrund ist zu schließen, dass die Reduktion von Zertifikaten sowohl heute als auch in Zukunft eine zentrale Rolle für eine schnelle, effiziente und weit verbreitete Kommunikation spielt.

Ziel dieser Arbeit ist es, Verfahren zur Reduktion der übertragenen X.509-Zertifikate(RFC 5280) in TLS-Verbindungen zu erläutern und miteinander zu vergleichen.

Für eine schnellere und effizientere TLS-Verbindung gibt es verschiedene Ansätze und Empfehlungen, wie beispielsweise die Nutzung von TLS 1.3 statt TLS 1.2, der Einsatz von Elliptic Curve Cryptography (ECC) anstelle von RSA, die Optimierung der Zertifikatskette, OCSP-Stapling, die Verwendung von Hardware-Kryptografie und weitere Maßnahmen.

In dieser Arbeit werden ausschließlich die folgenden Ansätze betrachtet:

- Zertifikatskompression (RFC 8879)
- Cached Information Extension (RFC 7924)
- Client Certificate URLs (RFC 6066)
- CBOR-encoded Certificates (I-D.ietf-cose-cbor-encoded-cert)
- Raw Public Keys (RFC 7250)
- cTLS Certificate Handles (I-D.ietf-tls-ctls)

Der Vergleich erfolgt anhand von drei Kriterien:

- Dem möglichen Grad der Reduktion der zu übertragenden Zertifikate,
- Der Kompatibilität des Ansatzes mit bestehenden TLS-Stacks,
- Der Bewertung des Ansatzes im Hinblick auf Sicherheit.

## **2 Erläuterung der Ansätze**

In diesem Kapitel wird die grundlegende Funktionsweise der in der Einleitung genannten Ansätze zur Reduktion der Zertifikatsgröße erläutert. Anschließend wird der mögliche Grad der Reduktion der zu übertragenden Zertifikate untersucht und die Sicherheit des Ansatzes kritisiert.

### **2.1 Zertifikatskompression (RFC 8879)**

#### **Funktionsweise**

In diesem Ansatz einigen sich die Kommunikationspartner während des TLS 1.3-Handshakes auf einen gemeinsamen Kompressionsalgorithmus für die Übertragung der Zertifikatskette. Anschließend wird die Zertifikatskette mit diesem Algorithmus komprimiert und in einer speziellen Compressed Certificate-Struktur gesendet, die die normale Certificate-Nachricht ersetzt.

Mögliche Kompressionsalgorithmen gemäß RFC 8879 sind:

- zlib (Algorithmus-ID: 1)
- Brotli (Algorithmus-ID: 2)
- zstd (Algorithmus-ID: 3)

## **Einschränkungen**

Diese Erweiterung wird nur mit TLS 1.3 (RFC8446) unterstützt.

## **Unterstützte TLS-Stacks**

OpenSSL, BoringSSL, Rustls, GnuTLS

## **Sicherheitsaspekte**

Der TLS-Stack muss beim Padding (RFC 8446, Abschnitt 5.4) die Kompression berücksichtigen. Beim Padding wird die Zertifikatskette auf eine einheitliche Endlänge aufgefüllt, damit ein Angreifer aus der Länge des verschlüsselten Zertifikats keine Rückschlüsse ziehen kann. Wenn beim Padding nicht auf den eingesetzten Kompressionsalgorithmus geachtet wird, kann das Padding nicht wie vorgesehen wirken.

RFC 7525 empfiehlt, Kompression auf der TLS-/DTLS-Ebene zu deaktivieren, da sie anfällig für Angriffe wie CRIME<sup>1</sup> ist. Für IoT-Anwendungen ist eine Kompression auf der TLS-/DTLS-Schicht nicht erforderlich, da Protokolle auf Anwendungsebene bereits stark optimiert sind und Kompressionsalgorithmen auf der DTLS-Ebene den Codeumfang und die Komplexität erhöhen.

## **Erzielbaren Größenreduktion**

Es wird gemessen, wie groß die unkomprimierte und die komprimierte Zertifikatskette von www.example.com sind. Die Komprimierung wird auf das DER-Format angewendet. Ziel der Untersuchung ist es, die erzielbare Größenreduktion verschiedener Algorithmen zu quantifizieren.

---

<sup>1</sup> CRIME ist ein Seitenkanalangriff, der die Längenreduktion durch Kompression in TLS ausnutzt, um aus der Größe verschlüsselter Nachrichten Rückschlüsse auf geheime Inhalte zu ziehen.

die angewendete Zertifikatskette:

| Name des Inhabers                                   | Öffentlicher Schlüssel Algorithmus          | öffentlicher Schlüssel | Schlüssellänge | Signatur |
|-----------------------------------------------------|---------------------------------------------|------------------------|----------------|----------|
| *.example.com                                       | Elliptische Kurve<br>Öffentlicher Schlüssel | 65 Byte                | 256 Bit        | 103 Byte |
| DigiCert<br>Global G3 TLS<br>ECC SHA384<br>2020 CA1 | Elliptische Kurve<br>Öffentlicher Schlüssel | 97 Byte                | 384 Bit        | 103 Byte |
| DigiCert<br>Global Root G3                          | Elliptische Kurve<br>Öffentlicher Schlüssel | 65 Byte                | 384 Bit        | 103 Byte |

Die **unkomprimierte** Dateigröße der Zertifikatskette von www.example.com ist **1,4KB**.

mit **zlib komprimierte** Dateigröße der Zertifikatskette von www.example.com ist **1,1KB**  
**Reduktion: 21%**

mit **brotili --quality=5 komprimierte** Dateigröße der Zertifikatskette von www.example.com ist **1,1KB**  
**Reduktion: 21%**

mit **brotili --quality=10 komprimierte** Dateigröße der Zertifikatskette von www.example.com ist **1KB**  
**Reduktion: 28%**

mit **zstd --quality=9 komprimierte** Dateigröße der Zertifikatskette von www.example.com ist **1,1KB**  
**Reduktion: 21%**

mit **zstd --quality=19 komprimierte** Dateigröße der Zertifikatskette von www.example.com ist **1,1KB**  
**Reduktion: 21%**

## 2.2 Client Certificate URLs (RFC 6066)

### Funktionsweise

Dieses Verfahren ermöglicht es dem Client, anstelle seines vollständigen X.509-Zertifikats lediglich eine URL zu diesem Zertifikat sowie den zugehörigen SHA-1-Hash in der Certificate-Nachricht zu übermitteln. Der Server ruft die eigentlichen Zertifikate anschließend selbst über die angegebenen URLs ab. Auf diese Weise reduziert der Client die Übertragungszeit und zu übermittelnde Datenvolumen.

### Einschränkungen

Die Extension ist in TLS 1.3 nicht mehr erlaubt.

### Unterstützte TLS-Stacks

Übliche TLS-Stacks wie OpenSSL, BoringSSL, Rustls und GnuTLS unterstützen diesen Ansatz nicht.

### Sicherheitsaspekte

Der Server muss die angegebenen URLs aktiv abrufen. Da dieser Abruf vor jeder Client-Authentifizierung erfolgt, erzeugt es mehrere Angriffsvektoren wie Server-Side Request Forgery, Denial-of-Service, Distributed DoS.

In diesem Ansatz ist SHA-1 als feste Standard-Hashfunktion vorgesehen, was aufgrund bekannter kryptographischer Schwächen heute als unsicher gilt.

#### Beispiel Szenario 1:

Ein bössartiger Client schickt dem Server die URL: <http://127.0.0.1:8080/admin/deleteAll>

Der TLS-Server führt den Abruf aus .

Der Angreifer zwingt den Server, interne Systeme anzugreifen.

#### Beispiel Szenario 2:

Der Client gibt eine URI an, die auf denselben TLS-Server verweist (z. B. <https://example.com>). Der Server ruft sich selbst auf und führt dabei intern einen vollständigen TLS-Client-Handshake mit sich selbst durch. Er spielt die Rolle des Clients.

Da der Server in diesem internen Handshake erneut eine `cached_info`-Extension vorfindet, entsteht eine Endlosschleife, die zu einer 100% Auslastung von CPU und Arbeitsspeicher führen kann.

### Erzielbaren Größenreduktion

Der Client reduziert zwar Übertragungszeit und Datenvolumen, jedoch entstehen für den Server zusätzliche Kosten, da jede URL einen weiteren Netzwerkabruf wie DNS-Lookup (RFC 1034), TCP-Handshake (RFC 793), TLS-Handshake (RFC 5246) und HTTP-GET (RFC 9110) erfordert.

## 2.3 Cached Information Extension (RFC 7924)

### Funktionsweise

Beschreibt einen Mechanismus, mit dem ein Client und ein Server die Übertragung von Zertifikaten vermeiden können, die bereits in einem früheren Handshake ausgetauscht wurden. Dies hilft jedoch nicht, wenn der Client zum ersten Mal eine Verbindung zu einem Server herstellt.

Beim nächsten Besuch von genau diesem Server, sendet Der Client im *ClientHello* eine neue Extension namens *cached\_info*.

Darin teilt er dem Server mit, welche Zertifikate er bereits lokal gecacht hat, und zwar nur einen kryptografischen Hash (standardmäßig SHA-256) dieser Daten, nicht die Daten selbst. Erkennt der Server anhand des Hashes, dass er exakt dieselben Daten verwenden würde, kann er sie im Handshake einfach weglassen.

Der Handshake wird dadurch deutlich kleiner und schneller.

Die Gründe dafür, dass der vom Client übermittelte Zertifikat-Hash nicht mehr mit dem aktuellen Serverzertifikat übereinstimmt, könnten unter anderem einer regulären Zertifikaterneuerung, einer Schlüsselrotation des Servers, eines Zertifikatswiderrufs und der Neuausstellung oder durch die Verwendung unterschiedlicher Zertifikate in verteilten Serverinfrastrukturen sein.

### Einschränkungen

Die Extension ist in TLS 1.3 nicht mehr erlaubt.

### Unterstützte TLS-Stacks

Übliche TLS-Stacks wie OpenSSL, BoringSSL, Rustls und GnuTLS unterstützen diesen Ansatz nicht mehr.

### Sicherheitsaspekte

Datenschutzverletzungen durch Fingerprinting sind das größte Problem. Der Client sendet die Hashes aller zwischengespeicherten Zertifikate in einem unverschlüsselten ClientHello. Ein passiver Angreifer kann diese Hashes sehen und daraus ableiten, mit welchen Servern der Client verbunden war.

Clients müssen sicherstellen, dass sie Informationen nur aus echten, erfolgreichen TLS-Verbindungen speichern.

Wenn sie das nicht tun, kann ein Angreifer falsche Information in den Cache einschleusen.



**Beispiel Szenario:**

Angreifer stellt sich zwischen Client und Server.

Er fängt den ersten TLS-Handshake ab.

Der Angreifer sendet dem Client gefälschte *cached-Info-Daten* z. B. ein manipuliertes Zertifikat.

Wenn der Client das sofort speichert, ohne zu verifizieren, denkt er später, dass er dieses Zertifikat kennt, und bei der nächsten Verbindung benutzt der Client das *Cached-Info-Objekt*.

**Erzielbaren Größenreduktion**

Der SHA-256-Hash der oben genannten Zertifikatskette von example.com ist  
`455943cf819425761d1f950263ebf54755d8d684c25535943976f488bc79d23b`.

Ein SHA-256-Hash hat immer eine feste Größe von 32 Byte. Die oben genannte Zertifikatskette von example.com hingegen ist etwa 1,4 KB groß.

**Der Server erkennt den Hash**

Der Client sendet den SHA-256-Hash der gecachten Zertifikate. Erkennt der Server den Hash, bestätigt er dies, indem er denselben Hash zurücksendet.

**Der Server erkennt den Hash nicht**

Der Client sendet den SHA-256-Hash der gecachten Zertifikate. Erkennt der Server den Hash nicht, übermittelt er die vollständige Zertifikatskette. Der Client aktualisiert danach seinen Cache.

**2.4 CBOR-encoded Certificates (I-D.ietf-cose-cbor-encoded-cert)****Funktionsweise**

CBOR(RFC 7049) steht für Concise Binary Object Representation. ES ist ein binäres Datenformat und entwickelt für IoT, Embedded Systems, und Protokolle mit wenig Bandbreite. Sie ersetzt die ASN.1(RFC 6025).

CBOR-encoded Certificates, stellen eine kompakte Codierung von X.509-Zertifikaten in Concise Binary Object Representation dar. Sie reduzieren der Größe und Komplexität von Zertifikaten, insbesondere in ressourcenbeschränkten Umgebungen wie IoT-Anwendungen.

Die Codierung unterstützt einen großen Subset der in RFC 5280 definierten X.509-Strukturen und ist vollständig kompatibel mit Profilen wie RFC 7925, IEEE 802.1AR (DevID), CNSA 1.0, RPKI, GSMA eUICC.

Es gibt 2 Typen. Invertierbare Re-Codierung (Typ 3) und Nativ signierte C509-Zertifikate (Typ 2).

Typ 3 ist, ein bereits vorhandenes DER-codiertes X.509-Zertifikat in das CBOR-Format umzuwandeln.

Typ 2 ist, die Signatur direkt über CBOR berechnen, ohne DER-Zwischenschritt. Das ist noch effizienter, aber nicht rückwärtskompatibel.

## **Einschränkungen**

Keine Unterstützung für BER<sup>1</sup> codierte Zertifikate; nur DER<sup>2</sup> ist kompatibel.

## **Unterstützte TLS-Stacks**

Es ist noch ein IETF-Standard-Entwurf (Internet-Draft). Daher gibt es noch keine breite Unterstützung in den TLS-Stacks Mbed TLS und OpenSSL.

Herkömmliche Webbrowser oder Server können CBOR-Zertifikate ohne spezielle Software-Updates nicht verarbeiten.

## **Sicherheitsaspekte**

C509 erbt die Sicherheitsannahmen von RFC 5280, ohne zusätzliche Informationen preiszugeben, die über X.509 hinausgehen. Bietet zusätzliche Sicherheitsgarantien.

Es wurde demonstriert, dass CBOR-Formate mit verifizierten Parser-Kombinatoren implementiert werden können, die adversarielle Eingaben handhaben und Non-Malleability sicherstellen. (T. Ramananandro et al., 2025).

Die Größenreduktion könnte die Verwendung von C509 erkennbar machen.

---

<sup>1</sup>Basic Encoding Rules

<sup>2</sup>Distinguished Encoding Rules

## Erzielbaren Größenreduktion

Wenn die CBOR-Kodierung verwendet wird, um DER-kodierte X.509-Zertifikate neu zu kodieren, kann die Größe von RFC-7925-konformen Zertifikaten in vielen Fällen um mehr als 50 % reduziert werden (vgl. Preuß Mattsson et al., 2026).

RFC 7925 legt fest, wie man TLS und DTLS so konfiguriert, dass sie auf ressourcenbeschränkten Geräten (Sensoren, Smart Home, Industrie-IoT) effizient funktionieren.

RFC 7925 konforme Zertifikate sind Zertifikate, die den speziellen Sicherheits- und Strukturregeln für diese IoT Geräte entsprechen.<sup>1</sup>

Durch die Kombination der CBOR-Kodierung anstelle von ASN.1 sowie unter Ausnutzung der Einschränkungen des XIOT-Profiles kann die Größe von IoT-Zertifikaten um mehr als 50 % reduziert werden (vgl. Höglund et al. 2020).

## 2.5 Raw Public Keys (RFC 7250)

### Funktionsweise

Normalerweise sendet ein Server im TLS-Handshake ein vollständiges X.509-Zertifikat(RFC 5280). Ein Raw Public Key(RFC 7250) hingegen ist nur der reine kryptografische Schlüssel ohne die üblichen Metadaten wie Aussteller, Inhaber oder Gültigkeitsdauer.

Client und Server signalisieren im *ClientHello* über die Erweiterung *client\_certificate\_type* oder *server\_certificate\_type*, dass sie RPK unterstützen.

Statt der Nachricht *Certificate* mit einer komplexen Kette wird nur der öffentliche Schlüssel gesendet.

Endpunkte speichern lokale Tabellen, die Identifikatoren (z. B. Hostnamen oder IPs) mit vertrauenswürdigen öffentlichen Schlüsseln verknüpfen oder Clients überprüfen Server-RPKs mittels TLSA-Resource-Records in DNSSEC-geschütztem DNS (DANE RFC 6698).

---

<sup>1</sup>Diese Ansatz nennt man Profiling. Profiling ist die Anpassung und Einschränkung von Zertifikatsinhalten und -strukturen, um sie für spezifische Anwendungsumgebungen zu optimieren.

## Einschränkungen

In einem offenen System (wie dem World Wide Web) ist RPK kaum nutzbar, da man nicht alle öffentlichen Schlüssel der Welt vorab speichern kann.

Keine automatische Validierung über CAs; eignet sich für geschlossene Systeme oder wo Identität anders gesichert ist z. B. via DANE(RFC 6698) oder Pre-Shared Keys

## Unterstützte TLS-Stacks

Übliche TLS-Stacks wie OpenSSL, BoringSSL, wolfSSL und GnuTLS unterstützen diesen Ansatz.

## Sicherheitsaspekte

Es wird empfohlen, diesen Ansatz nur in geschlossenen Netzwerken zu verwenden.

Da keine Signatur einer Certificate Authority vorliegt, muss die Bindung zwischen dem Schlüssel und der Identität des Geräts vorab bekannt oder durch Mechanismen wie DANE (RFC 6698) erfolgen.

Möglicher Angriffe sind z.B. Identity-Misbinding-Attacks<sup>2</sup> (Moustafa et al., 2024) .

## Erzielbaren Größenreduktion

Statt der Nachricht Certificate mit einer komplexen Kette wird nur der öffentliche Schlüssel gesendet. Dateigröße der Zertifikatskette von `www.example.com` ist 1,4 KB.

nur der öffentliche Schlüssel von `www.example.com` ist 65 Byte.

---

<sup>2</sup>Angreifer können Identitäten falsch binden, z. B. indem ein bössartiger Domain-Besitzer den Hash eines ehrlichen Server-Schlüssels in seinen eigenen TLSA-Records registriert. Clients validieren erfolgreich, verbinden sich aber mit dem falschen Server via IP-Umleitung. Ähnliche Angriffe treten bei vorkonfigurierten Schlüsseln auf, z. B. in IoT-Szenarien mit Duplikat-Schlüsseln und Netzwerkumleitungen

## 2.6 cTLS Certificate Handles (I-D.ietf-tls-ctls)

### Funktionsweise

cTLS (Compact TLS 1.3) ist eine optimierte Variante von TLS 1.3. In einem Standard-TLS-Handshake sendet der Server seine gesamte Zertifikatskette. Bei cTLS Certificate Handles wird die Zertifikatskette durch einen ID ersetzt.

Der Empfänger nutzt diesen ID, um das vollständige Zertifikat in seinem lokalen Speicher nachzuschlagen.

### Einschränkungen

Beide Kommunikationspartner müssen das Zertifikat und ID bereits kennen.

### Unterstützte TLS-Stacks

cTLS ist noch ein experimentelles IETF-Draft-Verfahren. Weit verbreitete Bibliotheken wie OpenSSL unterstützen cTLS derzeit nicht.

WolfSSL berichtet dass man die Standardisierung von cTLS aktiv verfolgt (wolfSSL 2022).

### Sicherheitsaspekte

Andere Server wissen nicht, ob eine ID bereits vergeben ist. Daher müssen die IDs eindeutig sein. Dafür könnte beispielsweise der Hash des originalen Zertifikats als ID verwendet werden. Alternativ können in großen Netzwerken gemeinsame Listen eingesetzt werden, auf die jeder Server im Netzwerk Zugriff hat.

### Erzielbaren Größenreduktion

In einem Standard-TLS-Handshake sendet der Server seine gesamte Zertifikatskette. Sie ist z.B. 1,4KB groß in example.com. Bei cTLS Certificate Handles wird die Zertifikatskette durch einen ID ersetzt. Diese ID ist nur 1–255 Bytes.

## 4 Fazit

Zusammenfassend lässt sich feststellen, dass die Reduktion der Zertifikatsgröße in TLS-Verbindungen ein zentrales Thema für die Optimierung von Netzwerkressourcen darstellt, insbesondere in ressourcenbeschränkten Umgebungen wie IoT-Geräten oder drahtlosen Netzwerken. Die in dieser Arbeit untersuchten Ansätze bieten vielfältige Methoden zur Verringerung des Übertragungsvolumens, weisen jedoch unterschiedliche Stärken und Schwächen in Bezug auf Reduktionsgrad, Kompatibilität mit TLS-Stacks, Sicherheit, Implementierungsaufwand und Eignung für große / dynamische Netzwerke auf. Ein Vergleich der Ansätze anhand der festgelegten Kriterien zeigt folgendes Bild:

*Alle Größen- und Reduktionsangaben beziehen sich, sofern nicht anders angegeben, auf die 1,4 KB große Zertifikatskette von [www.example.com](http://www.example.com).*

| Ansatz                                    | Reduktion                                                                                        | Kompatibilität mit TLS-Stacks             | Bewertung von Sicherheit                                             | Aufwand von Implementation                                                                                      | Eignung für große/ dynamische Netzwerke                                                                |
|-------------------------------------------|--------------------------------------------------------------------------------------------------|-------------------------------------------|----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>Zertifikatskompression (RFC 8879)</b>  | Niedrig (bis zu 1KB)                                                                             | Hoch (OpenSSL, BoringSSL, Rustls, GnuTLS) | Hoch (Risiken durch Padding-Ineffizienz und CRIME-ähnliche Angriffe) | Niedrig (Integration von Kompressionsalgorithmen in bestehende Stacks möglich, aber Konfiguration erforderlich) | Hoch (Dynamisch ausgehandelt, skalierbar für hohe Bandbreite-Reduktion in großen Netzen)               |
| <b>Client Certificate URLs (RFC 6066)</b> | Sehr hoch für Client (nur URL + SHA-1 Hash, 20 Bytes), aber Server-Abrufkosten ist ziemlich hoch | Niedrig (nicht mehr in TLS 1.3, veraltet) | Niedrig (SSRF, DoS, SHA-1-Schwächen)                                 | Hoch (Zusätzliche Logik für Abruf und Validierung, begrenzte Stack-Unterstützung)                               | Niedrig (Abruf-Overhead und Sicherheitsrisiken machen es ungeeignet für dynamische, skalierbare Netze) |

| Ansatz                                                             | Reduktion                                         | Kompatibilität mit TLS-Stacks                              | Bewertung von Sicherheit                                                           | Aufwand von Implementation                                                        | Eignung für große/dynamische Netzwerke                                                                        |
|--------------------------------------------------------------------|---------------------------------------------------|------------------------------------------------------------|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <b>Cached Information Extension (RFC 7924)</b>                     | sehr Hoch bei Folgehandshakes (nur 32-Byte-Hash)  | Niedrig (nicht mehr in TLS 1.3, veraltet)                  | Niedrig (Fingerprinting, Cache-Poisoning)                                          | Hoch (Wenig Implementierungen, erfordert Cache-Management)                        | Mittel (Gut für wiederholte Verbindungen in statischen Netzen, aber problematisch bei dynamischen Änderungen) |
| <b>CBOR-encoded Certificates (I-D.ietf-cose-cbor-encoded-cert)</b> | Mittel (bis zu 50 % für IoT-konforme Zertifikate) | Niedrig (Draft-Status, begrenzte Implementierungen)        | Hoch (Erbt X.509-Sicherheit, keine neuen Risiken)                                  | Mittel (CBOR-Encoding/Decoding hinzufügen)                                        | Hoch (Effizient für bandbreitenbeschränkte Netze, aber erfordert Kompatibilität in dynamischen Umgebungen)    |
| <b>Raw Public Keys (RFC 7250)</b>                                  | Sehr hoch (nur 65 Bytes Schlüssel)                | Hoch (OpenSSL, BoringSSL, wolfSSL, GnuTLS)                 | Mittel (Identity-Misbinding in offenen Netzen, empfohlen für geschlossene Systeme) | Mittel (Out-of-band-Bindung konfigurieren, Implementierungen in Stacks vorhanden) | Niedrig (Nicht skalierbar für offene/dynamische Netze ohne DANE, gut für geschlossene)                        |
| <b>cTLS Certificate Handles (I-D.ietf-tls-ctls)</b>                | Sehr hoch (1–255 Bytes ID)                        | Niedrig (experimentell, wolfSSL verfolgt Standardisierung) | Hoch (ID-Kollisionsrisiken, Privacy-Probleme durch Profile-IDs)                    | Hoch (Neue TLS-Variante, erfordert Anpassungen und ID-Management)                 | Mittel (Optimiert für constrained Netze, aber Vorkonfigurationen limitiert Dynamik in großen Netzen)          |

## Literaturverzeichnis

- [1] Forsby, Filip, et al. "Lightweight x. 509 digital certificates for the internet of things." *International Conference on Interoperability in IoT*. Cham: Springer International Publishing, 2017.
- [2] Kampanakis, Panos, et al. "The viability of post-quantum X. 509 certificates." *Cryptology ePrint Archive* (2018).
- [3] Sowa, Jakub, et al. "Post-quantum cryptography (pqc) network instrument: Measuring pqc adoption rates and identifying migration pathways." *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*. Vol. 1. IEEE, 2024.
- [4] Bodenhausen, Jörn, et al. "Bidirectional TLS Handshake Caching for Constrained Industrial IoT Scenarios." *2025 IEEE 50th Conference on Local Computer Networks (LCN)*. IEEE, 2025.
- [5] Tim Dierks, Eric Rescorla, The Transport Layer Security (TLS) Protocol Version 1.2, datatracker.ietf.org, August 2008, <https://datatracker.ietf.org/doc/html/rfc5246>, letzter Zugriff: 16.11.2025
- [6] Stefan Santesson, Hannes Tschofenig, Transport Layer Security (TLS) Cached Information Extension, datatracker.ietf.org, July 2016, <https://datatracker.ietf.org/doc/html/rfc7924>, letzter Zugriff: 16.11.2025
- [7] Donald Eastlake, Transport Layer Security (TLS) Extensions: Extension Definitions, datatracker.ietf.org, January 2011, <https://datatracker.ietf.org/doc/html/rfc6066>, letzter Zugriff: 16.11.2025
- [8] Alessandro Ghedini, Transport Layer Security (TLS) Certificate Compression draft-ietf-tls-certificate-compression-00, datatracker.ietf.org, June 08, 2017, <https://datatracker.ietf.org/doc/html/draft-ietf-tls-certificate-compression-00>, letzter Zugriff: 14.11.2025
- [9] Yoav Nir, Rich Salz, Nick Sullivan, Transport Layer Security (TLS) Extensions, iana.org, 2025-11-05, <https://www.iana.org/assignments/tls-extensiontype-values/tls-extensiontype-values.xhtml>, letzter Zugriff: 16.11.2025
- [10] OpenSSL announces final release of OpenSSL 3.2.0, openssl-library.org, November 23, 2023, <https://openssl-library.org/post/2023-11-06-openssl32>, letzter Zugriff: 15.11.2025
- [11] Rudolf Polzer, boringssl.googlesource.com, Nov 17, 2025, <https://boringssl.googlesource.com/boringssl/+HEAD/include/openssl/ssl.h>, letzter Zugriff: 15.11.2025
- [12] Rustls - a modern TLS Library, docs.rs, <https://docs.rs/rustls/latest/rustls/compress/index.html>, letzter Zugriff: 15.11.2025
- [13] Tim Rühsen, Daiki Ueno, Dmitry Baryshkov, Zoltán Fridrich, The GnuTLS Transport Layer Security Library, gnutls.org, <https://gnutls.org>, letzter Zugriff: 15.11.2025
- [14] John Preuß Mattsson, et al. CBOR Encoded X.509 Certificates, datatracker.ietf.org, January 08, 2026, <https://datatracker.ietf.org/doc/draft-ietf-cose-cbor-encoded-cert/>, letzter Zugriff: 11.01.2026
- [15] Hannes Tschofenig, et al. Transport Layer Security (TLS) / Datagram Transport Layer Security (DTLS) Profiles for the Internet of Things, datatracker.ietf.org, July, 2016, <https://datatracker.ietf.org/doc/html/rfc7925>, letzter Zugriff: 11.01.2026



- [16] John Preuß Mattsson, et al. CBOR Object Signing and Encryption (COSE): Headers for Carrying CBOR Compressed Certificates draft-mattsson-cose-cbor-cert-compress-00, datatracker.ietf.org, March 09, 2020, <https://datatracker.ietf.org/doc/html/draft-mattsson-cose-cbor-cert-compress-00>, letzter Zugriff: 11.01.2026
- [17] Höglund, Joel, et al. PKI4IoT: Towards public key infrastructure for the Internet of Things. Computers & Security, 2020, letzter Zugriff: 11.01.2026
- [18] Paul Wouters, Hannes Tschofenig, et al. Using Raw Public Keys in Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS), June 2014, <https://datatracker.ietf.org/doc/html/rfc7250>, letzter Zugriff: 12.01.2026
- [19] Paul Wouters, Hannes Tschofenig, et al. Using Raw Public Keys in Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS), June 2014, <https://datatracker.ietf.org/doc/html/rfc7250>, letzter Zugriff: 12.01.2026
- [20] Moustafa, Mariam, Mohit Sethi, and Tuomas Aura. "Misbinding Raw Public Keys to Identities in TLS." Nordic Conference on Secure IT Systems. Cham: Springer Nature Switzerland, 2024, [https://link.springer.com/chapter/10.1007/978-3-031-79007-2\\_4](https://link.springer.com/chapter/10.1007/978-3-031-79007-2_4), letzter Zugriff: 12.01.2026
- [21] R. Barnes, H. Tschofenig, et al. Compact TLS 1.3, April 2024, <https://www.ietf.org/archive/id/draft-ietf-tls-ctls-10.html>, letzter Zugriff: 12.01.2026
- [22] wolfSSL, cTLS: Compact TLS, January 2022, <https://www.wolfssl.com/ctls-compact-tls>, letzter Zugriff: 12.01.2026
- [23] Tahina Ramananandro, et al. Secure Parsing and Serializing with Separation Logic Applied to CBOR, CDDL, and COSE, November 2025, <https://dl.acm.org/doi/10.1145/3719027.3765120>, letzter Zugriff: 12.01.2026